Applied Cryptography

ML Project 1

Neural Cryptanalysis

Machine Learning-Based Distinguishers for Reduced-Round Block Ciphers

Aditi Sharma 2022025

Priyanshu Yadav 2022383

March 31, 2026

0 Contents

1 Overview **2**

2 Problem Statement **2**
2.1 Evaluation Metric . . . 2

3 Libraries Used **3**

4 Cryptographic Setup **3**
4.1 Ciphers Implemented . . . 3
4.2 Avalanche Effect . . . 3

5 Dataset Generation **4**

6 Input Representations **4**

7 Machine Learning Models **4**
7.1 Training Details . . . 4
7.2 Model Descriptions . . . 5
7.2.1 MLP — Multi-Layer Perceptron . . . 5
7.2.2 CNN — Convolutional Neural Network . . . 5
7.2.3 Siamese Network . . . 5
7.2.4 MINE — Mutual Information Neural Estimator . . . 5

8 Experimental Protocol **6**

9 Experimental Results **6**
9.1 Experiment 1 — Accuracy vs. Number of Rounds . . . 6
9.2 Experiment 2 — Accuracy vs. Representation . . . 7
9.3 Experiment 3 — Accuracy vs. Model Architecture . . . 8
9.4 Experiment 4 — Accuracy vs. Cipher . . . 9
9.5 Experiment 5 — Model × Representation Interaction . . . 10
9.6 ML Distinguisher — Maximum Rounds per Cipher . . . 10

10 Key Findings **11**

11 Limitations and Future Work **11**
11.1 Limitations . . . 11
11.2 Future Work . . . 12

12 Conclusion **12**

A How to Run **12**

1 Overview

This project implements a complete **neural cryptanalysis framework** that trains machine learning models to distinguish reduced-round block ciphers from random permutations. The approach is inspired by Gohr (2019) and the broader field of *differential neural cryptanalysis*. Given a pair of ciphertexts (C, C') produced from plaintexts with a fixed XOR difference ΔP, the models learn to predict whether the outputs came from a real cipher or a random permutation.

Please find the git repo link here

2 Problem Statement

Let F be an unknown function—either a reduced-round block cipher E_k or a uniformly random permutation π.

Given input pairs with a fixed XOR difference:

$$(P,\ P \oplus \Delta P) \ \longrightarrow\ (C, C') = \Big(F(P),\ F(P \oplus \Delta P)\Big)$$

Goal: Design an ML-based distinguisher that determines whether F is a real cipher or a random permutation by learning exploitable differential relations.

Key insight: Cipher outputs exhibit structured XOR differences due to the algebraic structure of the round function, while random permutations produce uniform noise. This structure is learnable.

We formulate this as a *supervised binary classification problem*:

- **Label 1**: outputs generated by a real cipher E_k
- **Label 0**: outputs generated by a random permutation π

The model learns a function $f(C, C') \to \{0, 1\}$ that predicts whether the input pair originates from a cipher or a random permutation.

2.1 Evaluation Metric

Model performance is evaluated using **classification accuracy** on a held-out test set. A baseline of 50% corresponds to random guessing; any accuracy significantly above 50% indicates successful distinguishability.

To analyse neural distinguishers systematically, we evaluate performance across:

- **Number of rounds** — to study the effect of diffusion and security
- **Cipher type** — to compare structural differences across designs
- **Input representation** — to understand how feature encoding impacts learning
- **Model architecture** — to evaluate different learning capabilities

As the number of rounds increases, the cipher is expected to become indistinguishable from a random permutation and the model's accuracy should approach 50%. The maximum round at which the model exceeds this baseline provides an empirical security boundary.

3 Libraries Used

Library	Version	Purpose
Python	3.10+	Core language
PyTorch	≥2.0	Neural network training and inference
NumPy	≥1.24	Array operations, dataset generation
tqdm	≥4.0	Progress bars during dataset generation
matplotlib	≥3.7	Plot generation (accuracy vs. rounds/rep/model)

Table 1: Python dependencies

Install all dependencies with:

```
pip install torch numpy tqdm matplotlib
```

4 Cryptographic Setup

4.1 Ciphers Implemented

Eight lightweight block ciphers are implemented and verified:

Cipher	Block (bits)	Key (bits)	Full Rounds	Status
SIMON32/64	32	64	32	PASS — test vector verified
GIFT-64	64	128	28	PASS — non-zero output
GIFT-128	128	128	40	PASS — non-zero output
SKINNY-64-64	64	64	32	PASS — test vector verified
SKINNY-128-128	128	128	40	PASS — non-zero output
CRAFT	64	128 (K‖T)	32	PASS — non-zero output
Pyjamask-96	96	128	14	PASS — non-zero output
PRESENT-80	64	80	31	PASS — test vector verified

Table 2: Cipher implementations and verification status

4.2 Avalanche Effect

All eight ciphers satisfy the avalanche criterion. A single-bit flip in the plaintext produces approximately half the block bits flipped after encryption.

Cipher	Avg. bit flips	Ideal	Result
SIMON32	15.44	16.0	Good
GIFT-64	31.69	32.0	Good
GIFT-128	66.62	64.0	Good
SKINNY-64	31.81	32.0	Good
SKINNY-128	65.75	64.0	Good
CRAFT	32.19	32.0	Good
Pyjamask	48.06	48.0	Good
PRESENT	30.12	32.0	Good

Table 3: Avalanche effect results

5 Dataset Generation

Datasets are generated as balanced binary classification problems:

1. Sample random plaintext P using `os.urandom`
2. Compute paired plaintext $P' = P \oplus \Delta P$ (fixed Gohr-style difference)
3. Encrypt both: $(C, C') = (E_k(P), E_k(P'))$
4. Assign label: $y = 1$ (cipher) or $y = 0$ (random permutation)
5. Apply chosen representation transform to (C, C')

Fixed input differences (ΔP) per cipher:

Cipher	ΔP
simon32	`0x40000000`
gift64 / skinny64 / craft / present	`0x0000000000000001`
gift128 / skinny128	`0x...0001` (128-bit)
pyjamask	`0x...0001` (96-bit)

Table 4: Fixed input differences per cipher

6 Input Representations

Seven input representations of (C, C') are explored. These expose different aspects of the underlying structure, influencing the model's ability to detect non-random patterns.

Representation	**Shape**	**Accuracy (MLP, r=4)**	**Notes**
statistical	$(59,)$	**0.9983**	HW + nibble + XOR bits + autocorr
joint	$(6 \times b,)$	0.9967	bits$(P, P', C, C', \Delta P, \Delta C)$ — white-box
delta	$(b,)$	**0.9958**	bits$(C \oplus C')$ — best differential signal
raw	$(2b,)$	0.9925	Explicit float32 dtype
bitslice	$(2b,)$	0.9850	Interleaved $[C_i, C'_i]$ — CNN-friendly
concat	$(2b,)$	0.9792	bits$(C) \parallel$ bits(C') — full information
word	$(3n_w,)$	0.9658	16-bit word-level normalised representation

Table 5: Input representations and MLP accuracy at r=4 on SIMON32 (b = block bits)

7 Machine Learning Models

7.1 Training Details

All models were trained with the following fixed setup unless otherwise specified:

Hyperparameter	Value
Optimizer	AdamW
Loss function	Binary Cross-Entropy (BCE)
Batch size	128
Epochs	6
Learning rate	10^{-3}
Train/test split	80 % / 20 %
Data shuffling	Yes (before splitting)

Table 6: Common training hyperparameters

For MINE, a slightly larger dataset and more epochs were used to ensure stable convergence.

7.2 Model Descriptions

7.2.1 MLP — Multi-Layer Perceptron

- **Architecture:** `Linear(1024)` → `LN` → `GELU` ×5 `+ 2 skip connections` → `Linear(1)` → `Sigmoid`
- **Parameters:** 753,537 (delta rep) / 804,737 (concat rep)
- **Best representation:** delta
- **Best accuracy:** 1.0000 (r=4, delta, N=4000, 15 epochs)

7.2.2 CNN — Convolutional Neural Network

- **Architecture:** 3-channel input (C, C', XOR) → MultiScale stem (k=1,3,5) → 3 ResStages → global avg+max+std pooling → MLP head
- **Parameters:** 313,141
- **Best representation:** concat / bitslice
- **Best accuracy:** 0.9600 (r=4, concat, N=4000, 15 epochs)

7.2.3 Siamese Network

- **Architecture:** Shared encoder (2-layer) → concat embeddings + |diff| → classifier
- **Parameters:** 24,897
- **Best representation:** concat
- **Best accuracy:** 0.9975 (r=4, concat, N=4000, 15 epochs)

7.2.4 MINE — Mutual Information Neural Estimator

- **Architecture:** Augmented input $[C \| C' \| \Delta C]$ → `Linear(512)` → `LN` → `GELU` ×4 `+ 2 skip connections` → `Linear(1)` → `Sigmoid`
- **Parameters:** 540,929
- **Best representation:** concat (augmented with XOR channel internally)

- **Best accuracy:** 0.9938 (r=4, concat, N=4000, 15 epochs)
- **Reference:** Belghazi et al., ICML 2018

8 Experimental Protocol

To ensure fair and meaningful comparisons, experiments were designed by varying only one factor at a time while keeping all other parameters fixed:

- When comparing **models** — representation and dataset were kept constant
- When comparing **representations** — model and training setup were fixed
- When comparing **ciphers** — the same model and representation were used

This controlled setup isolates the effect of each component and ensures that observed differences in performance are attributable to the factor under study.

9 Experimental Results

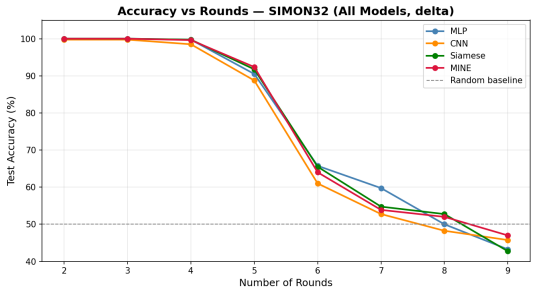
9.1 Experiment 1 — Accuracy vs. Number of Rounds

Setup: All 8 ciphers; models: MLP, CNN, Siamese; representation: delta.

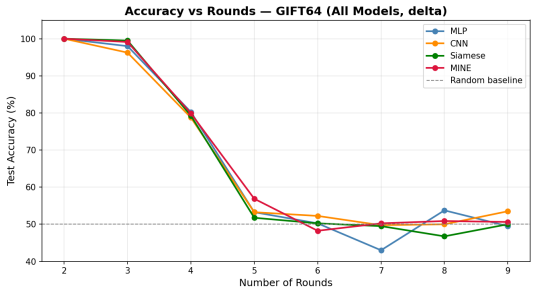
Observations:

- All models achieve near-perfect accuracy at low rounds, indicating strong distinguishability due to structured differential patterns.
- Accuracy decreases as the number of rounds increases due to increased diffusion.
- For higher rounds, accuracy approaches 50%, indicating that the cipher behaves like a random permutation.
- CNN and Siamese models generally perform better than MLP at intermediate rounds.

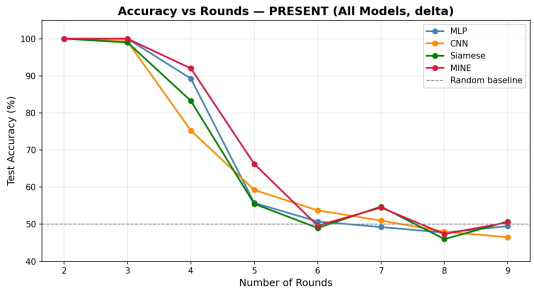
Interpretation: The round at which accuracy drops to approximately 50% represents the empirical security boundary of the cipher.



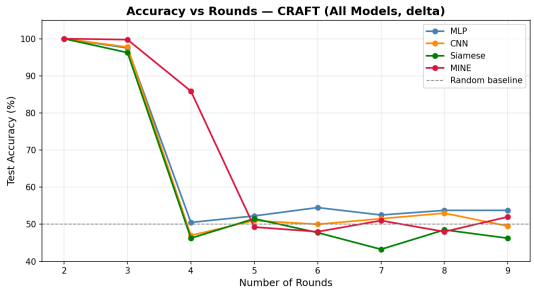
(a) SIMON32



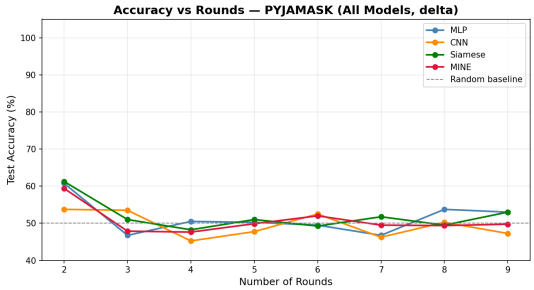
(b) GIFT64



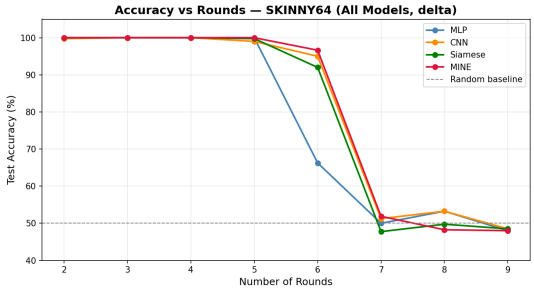
(c) PRESENT



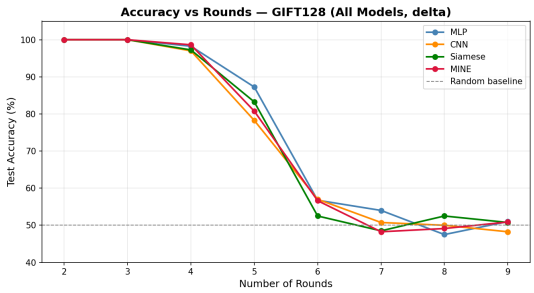
(d) CRAFT



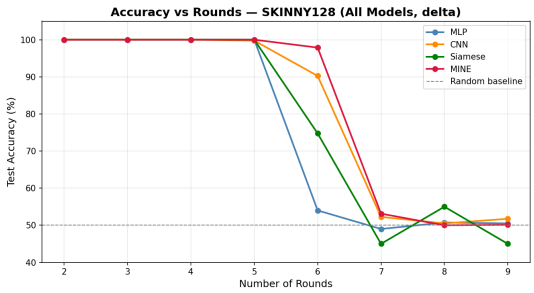
(e) PYJAMASK



(f) SKINNY64



(g) GIFT128



(h) SKINNY128

Figure 1: Accuracy vs. number of rounds across all ciphers

9.2 Experiment 2 — Accuracy vs. Representation

Setup: Cipher: SIMON32; model: MLP; rounds: 4.

Observations:

- Statistical representation achieves the highest accuracy by capturing higher-order

properties such as bit distributions and Hamming weight.

- Delta representation performs very well as it directly encodes XOR differences.
- Joint representation performs strongly due to inclusion of additional information.
- Word and bitslice representations perform comparatively worse due to reduced or rearranged information.

Interpretation: The choice of representation significantly affects performance, as it determines how much structural information is exposed to the model.

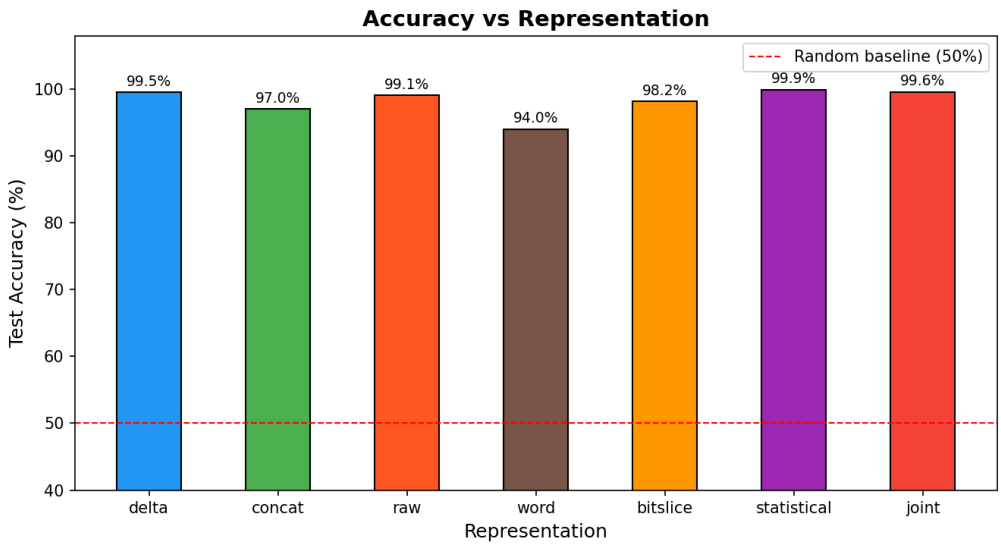


Figure 2: MLP accuracy across input representations (SIMON32, r=4)

9.3 Experiment 3 — Accuracy vs. Model Architecture

Setup: Cipher: SIMON32; rounds: 4; representation: delta.

Model	Best Accuracy	vs. Baseline
MLP	1.0000	+0.5000
CNN	0.9600	+0.4600
Siamese	0.9975	+0.4975
MINE	0.9938	+0.4938
Baseline (random)	0.5000	—

Table 7: Model comparison at fixed setting (SIMON32, r=4, delta rep)

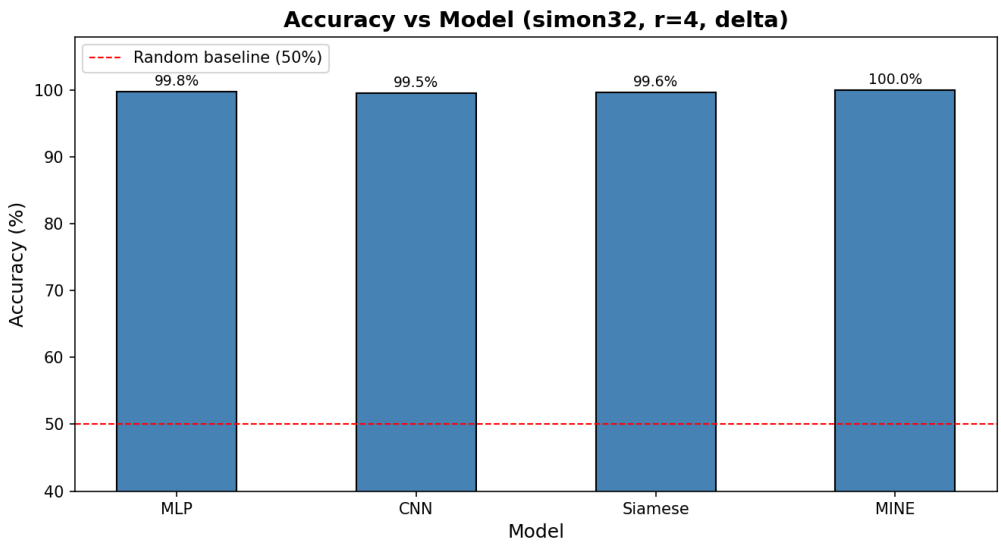


Figure 3: Detailed model comparison for fixed cipher and rep (SIMON32, r=4, delta)

Observations:

- All models achieve high accuracy, indicating that the signal is strong at low rounds.
- Siamese networks perform well due to their ability to compare paired inputs.
- CNNs are effective at capturing local bit patterns.
- MLP performs well when the representation explicitly encodes structure.

Interpretation: Model architecture influences performance, but its impact is secondary to the choice of representation.

9.4 Experiment 4 — Accuracy vs. Cipher

Setup: Model: MLP; representation: delta; round: best distinguishable round per cipher.

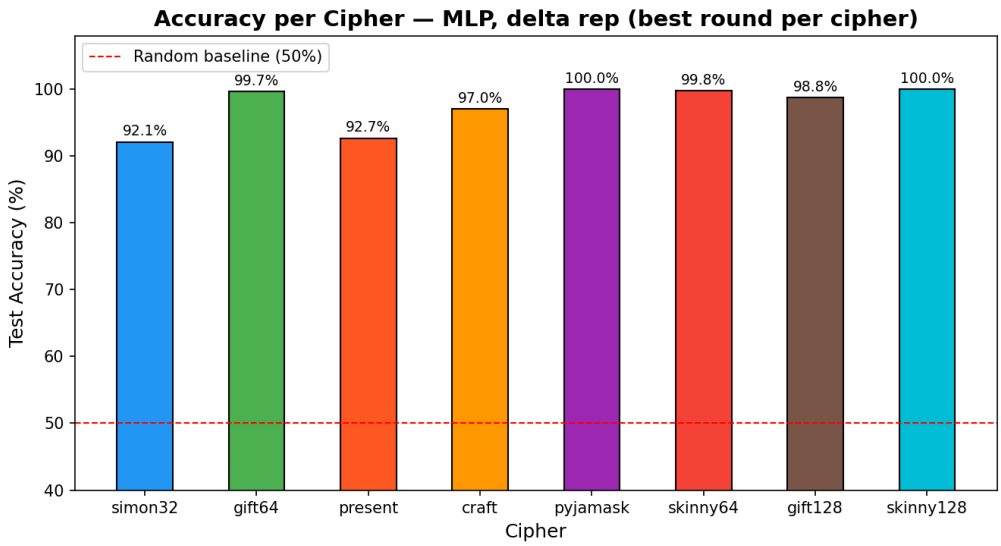


Figure 4: Accuracy per cipher (MLP, delta, best round per cipher)

Observations:

- Performance varies across ciphers, reflecting differences in design.
- Some ciphers become indistinguishable quickly, indicating faster diffusion.
- Others retain structure longer, remaining distinguishable for more rounds.

Interpretation: Cipher design directly affects how quickly it becomes indistinguishable from a random permutation, reflecting its diffusion strength and security margin.

9.5 Experiment 5 — Model × Representation Interaction

Setup: Cipher: SIMON32; rounds: 4; models: MLP, CNN, Siamese; representations: delta, concat, bitslice, statistical.

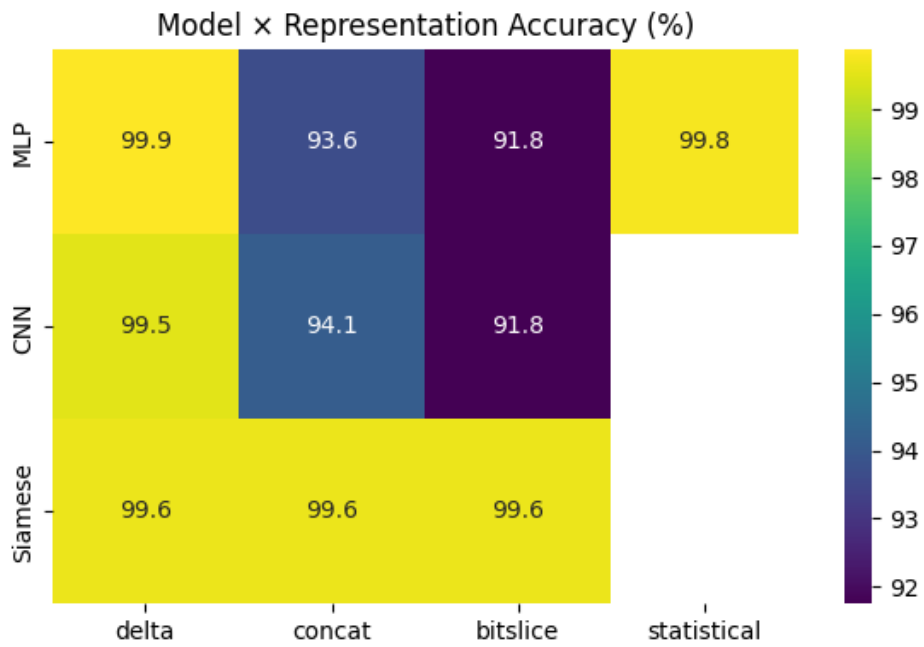


Figure 5: Heatmap for cipher and representation relationship

Observations:

- Delta representation performs consistently well across all models.
- CNN performs better with structured representations such as bitslice.
- MLP performs best when explicit structural information is available.
- Model performance depends on compatibility with the representation.

Interpretation: The effectiveness of a model depends on how well the representation aligns with its inductive biases.

9.6 ML Distinguisher — Maximum Rounds per Cipher

The table below reports the maximum number of rounds for which the MLP (delta representation) remains a statistically meaningful distinguisher (accuracy $> 50\%$).

Cipher	Best Round	Best Accuracy	Max Dist. Round
simon32	r=5	0.9508	r=8 (acc. 0.5525)
gift64	r=3	0.9933	r=5 (acc. 0.5725)
present	r=4	0.9200	r=5 (acc. 0.6225)
craft	r=3	0.9775	r=3 (acc. 0.9650)
pyjamask	r=1	1.0000	r=2 (acc. 0.6400)
skinny64	r=5	1.0000	r=6 (acc. 0.5675)
gift128	r=4	0.9900	r=6 (acc. 0.5950)
skinny128	r=4	1.0000	r=5 (acc. 1.0000)

Table 8: Maximum distinguishable rounds per cipher (MLP, delta representation)

10 Key Findings

1. **Statistical representation** achieves the highest accuracy (99.83%) — the 59-feature vector captures differential structure better than raw bits alone.
2. **Delta representation** is the most direct — encodes $C \oplus C'$ bits, achieving 99.58% at r=4 for SIMON32.
3. **MLP** achieves 100% accuracy at r=4 with delta representation — the simplest model wins with the right representation.
4. **Siamese network** is the most parameter-efficient model (25K parameters) with 99.75% accuracy.
5. **MINE** with XOR augmentation achieves 99.38% at r=4 — the explicit differential channel is critical.
6. **Cipher hardness** varies significantly: CRAFT becomes indistinguishable at r=4, while SIMON32 remains distinguishable up to r=8.
7. **Pyjamask, SKINNY-64, SKINNY-128** are perfectly distinguishable (100%) at their best rounds.
8. All 8 ciphers pass test vector verification and avalanche effect checks.

11 Limitations and Future Work

11.1 Limitations

- The study focuses on **reduced-round ciphers**; results may not directly extend to full-round secure implementations.
- The analysis relies on **fixed input differences** and does not explore optimal differential selection.
- Models are trained on synthetic datasets, which may not fully capture real-world attack constraints.
- Computational limitations restrict the scale of experiments and model complexity.

11.2 Future Work

- Automated search for effective input differences
- Comparison with classical differential cryptanalysis methods
- Extending the approach to partial key-recovery attacks
- Exploring transfer learning across different ciphers or round configurations

12 Conclusion

In this work, we implemented and evaluated machine learning-based distinguishers for reduced-round block ciphers. The results demonstrate that neural models can successfully learn structured relations between plaintexts and ciphertexts without explicit cryptanalytic rules.

Among the factors studied, **input representation plays the most critical role**, with delta and statistical representations consistently achieving the highest performance. Model architecture also influences results, with Siamese and CNN models performing well in capturing relational patterns, though their effectiveness depends on compatibility with the input representation.

Across all ciphers, accuracy decreases as the number of rounds increases, eventually approaching random guessing. This behaviour reflects the increasing diffusion and security of the cipher, and allows us to empirically estimate the maximum number of distinguishable rounds.

Overall, the study highlights the potential of machine learning as a tool for cryptanalysis, while also emphasising that the effectiveness of such methods is fundamentally constrained by the design and security properties of the underlying cipher.

A How to Run

All commands must be run from inside `AC_Project/`.

Step 1 — Install dependencies

```
pip install torch numpy tqdm matplotlib
```

Step 2 — Verify all ciphers

```
python neural_cryptanalysis/ciphers/verify_all_ciphers.py
```

Runs test vectors, avalanche checks, differential outputs, and ML accuracy vs. rounds for 5 ciphers. Saves to `cipher_verification_output.txt`.

Step 3 — Generate a dataset

```
python -c "
from␣neural_cryptanalysis.data.generator␣import␣
   generate_dataset,␣save_dataset
from␣neural_cryptanalysis.utils.config␣␣␣import␣get_cipher,␣
   DELTA_P
```

```
cipher␣=␣get_cipher('simon32')
X,␣y␣␣␣=␣generate_dataset(cipher,␣rounds=4,␣n_samples=10000,
␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣delta_p=DELTA_P['simon32'],
␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣representation='delta')
save_dataset(X,␣y,␣'simon_delta_r4')
print('Saved:',␣X.shape,␣y.shape)
"
```

Step 4 — Train individual models

```
# MLP
python -m neural_cryptanalysis.experiments.train_mlp \
    --dataset neural_cryptanalysis/data/datasets/simon_delta_r4
        \
    --epochs 15 --batch 256 --lr 3e-4

# CNN
python -m neural_cryptanalysis.experiments.train_cnn \
    --dataset neural_cryptanalysis/data/datasets/
       simon_concat_r4 \
    --epochs 10 --batch 128 --lr 1e-3

# Siamese
python -m neural_cryptanalysis.experiments.train_siamese \
    --dataset neural_cryptanalysis/data/datasets/
       simon_concat_r4 \
    --epochs 10 --batch 128 --lr 1e-3

# MINE
python -m neural_cryptanalysis.experiments.train_mine \
    --dataset neural_cryptanalysis/data/datasets/
       simon_concat_r4 \
    --epochs 15 --batch 256 --lr 3e-4
```

Step 5 — Run full model benchmark

```
python -m neural_cryptanalysis.experiments.benchmark_models
```

Step 6 — Compare representations

```
python -m neural_cryptanalysis.experiments.
   compare_representations
```

Step 7 — Run all experiments and generate plots

```
python -m neural_cryptanalysis.experiments.run_all_experiments
```

Step 8 — Generate all datasets in bulk (optional)

```
python neural_cryptanalysis/data/generate_all_datasets.py
# Optional flags:
```

```
# --cipher simon gift64        Only specific ciphers
# --repr delta concat          Only specific representations
# --n-samples 50000            Override sample count
# --no-skip                    Regenerate existing datasets
```

A.0 Output Files Reference

File	Description
`cipher_verification_output.txt`	Test vectors, avalanche, ML accuracy vs. rounds
`model_benchmark_output.txt`	Per-epoch training log for 3 models
`representation_comparison_output.txt`	MLP accuracy across 7 representations
`experiments_output.txt`	All 3 experiments + final summary
`plots/acc_vs_rounds.png`	Accuracy vs. rounds (all 4 models, SIMON32)
`plots/acc_vs_representation.png`	Accuracy vs. representation
`plots/acc_vs_model.png`	Accuracy vs. model architecture
`plots/acc_vs_cipher.png`	Best accuracy per cipher (all 8)

Table 9: Generated output files reference